

Python Visualization Library

Libraries Covered:

1. Matplotlib
2. Seaborn

1. Library Overview

1.1 Matplotlib

- **Overview:**
Matplotlib is the foundational Python library for 2D plotting. It's a powerful, low-level tool that gives complete control over every element of a figure.
- **Typical Use Cases:**
 - Scientific and academic plots
 - Custom static visualizations for reports or publications
 - Backend for higher-level visualization libraries (e.g., Seaborn)
- **Unique Features:**
 - Supports object-oriented and functional interfaces (`pyplot`)
 - Export to multiple formats (PDF, PNG, SVG)
 - Integration with GUIs (Tkinter, Qt, etc.) and LaTeX

1.2 Seaborn

- **Overview:**
Seaborn is a high-level statistical data visualization library built on top of Matplotlib. It simplifies complex visualizations and adds elegant styling by default.
- **Typical Use Cases:**
 - Visualizing statistical relationships
 - Creating aesthetically pleasing plots with less code
 - Working with Pandas DataFrames
- **Unique Features:**
 - Built-in themes and color palettes
 - Automatic aggregation and bootstrapping
 - Support for multi-plot grids and semantic mappings
 - Integration with Pandas and NumPy data structures

2. Library Comparison

Feature	Matplotlib	Seaborn
Ease of Use	Moderate: verbose, more manual	High: concise syntax, handles complexity under the hood

Feature	Matplotlib	Seaborn
Customization	Very high: full control over plot elements	Limited: relies on Matplotlib for advanced customization
Interactivity	Minimal: static plots by default	Same as Matplotlib; static
Performance	Excellent for small to medium datasets	Similar performance; may abstract some speed trade-offs
Data Handling	Manual data prep needed	Designed for Pandas; minimal prep
Plot Variety	Supports all classic plot types	Adds statistical and complex plots (e.g., violin, swarm)
Best For	Precision control, foundational library	Quick, beautiful, and statistical plots

Final Thoughts

- **Use Matplotlib** when you need full control and publication-ready precision.
- **Use Seaborn** when you want to explore or communicate patterns in data quickly with cleaner code and built-in statistical logic.

Matplot Lib

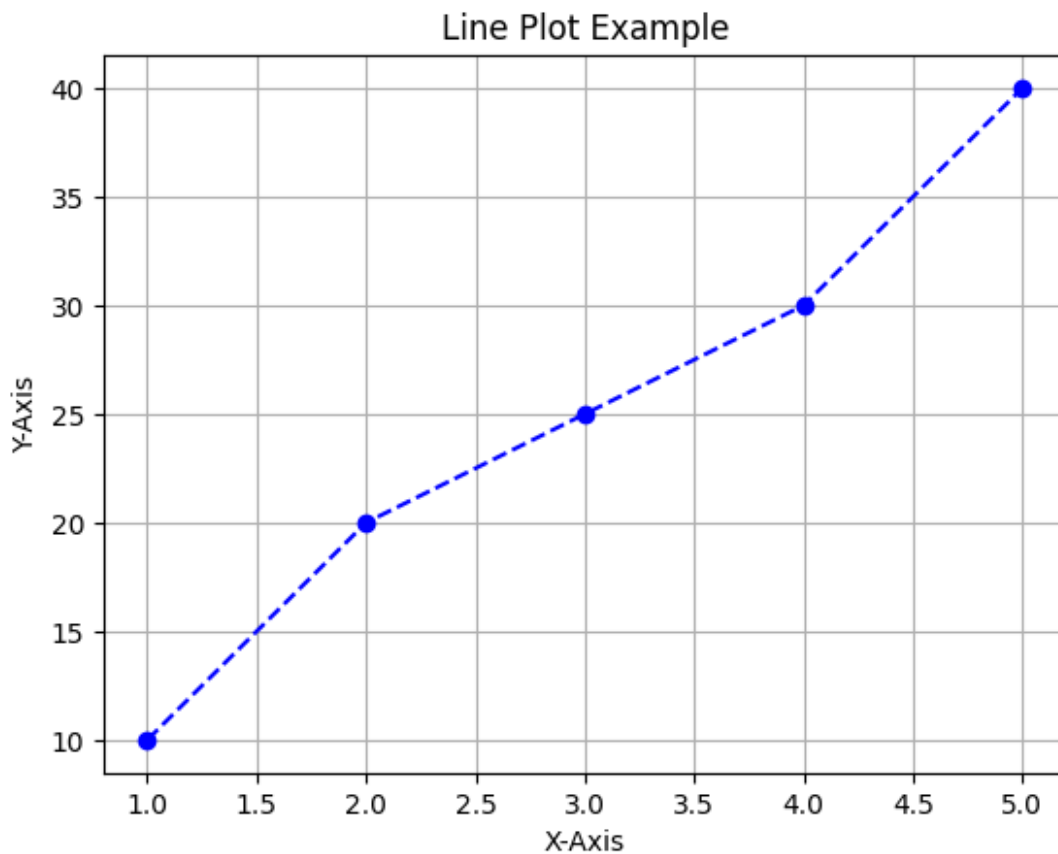
```
import matplotlib.pyplot as plt
import numpy as np
```

1. Line Plot Use for: Time series, trend analysis.

Function: plt.plot()

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

plt.plot(x, y, marker='o', color='blue', linestyle='--')
plt.title("Line Plot Example")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.grid(True)
plt.show()
```

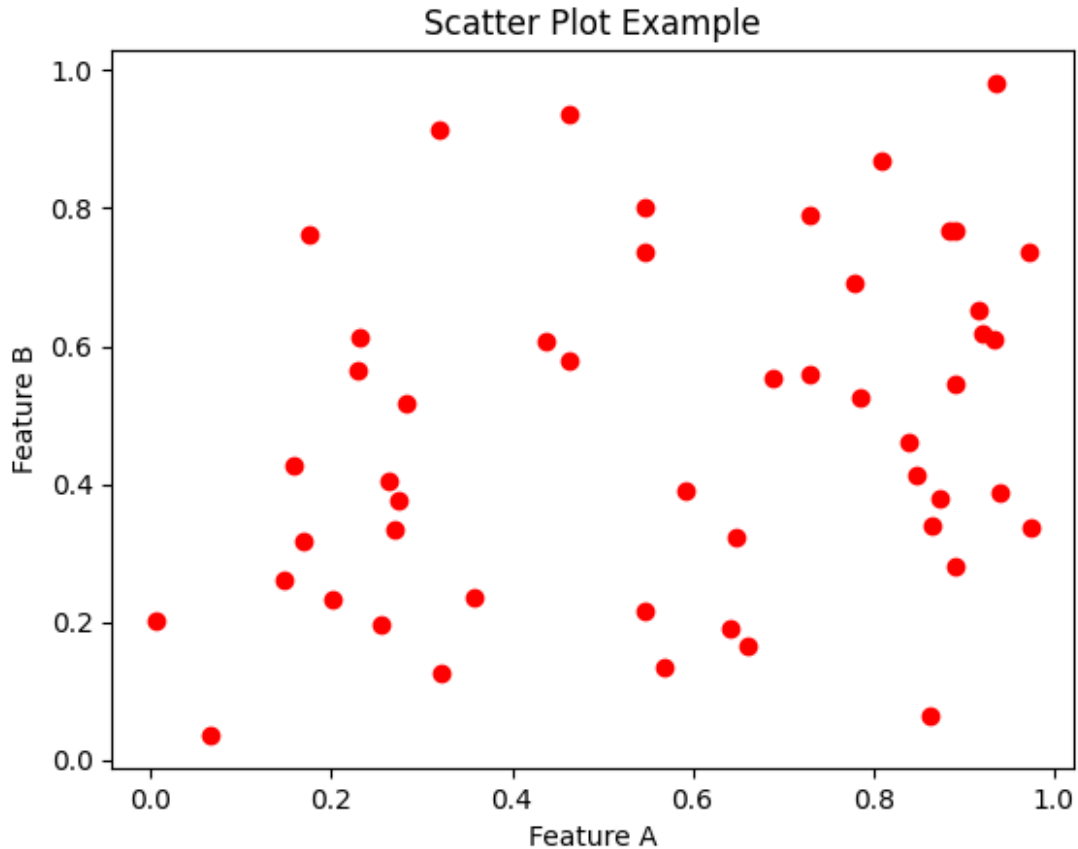


1. Scatter Plot Use for: Correlation, clustering, pattern analysis.

Function: plt.scatter()

```
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, color='red')
plt.title("Scatter Plot Example")
plt.xlabel("Feature A")
plt.ylabel("Feature B")
plt.show()
```

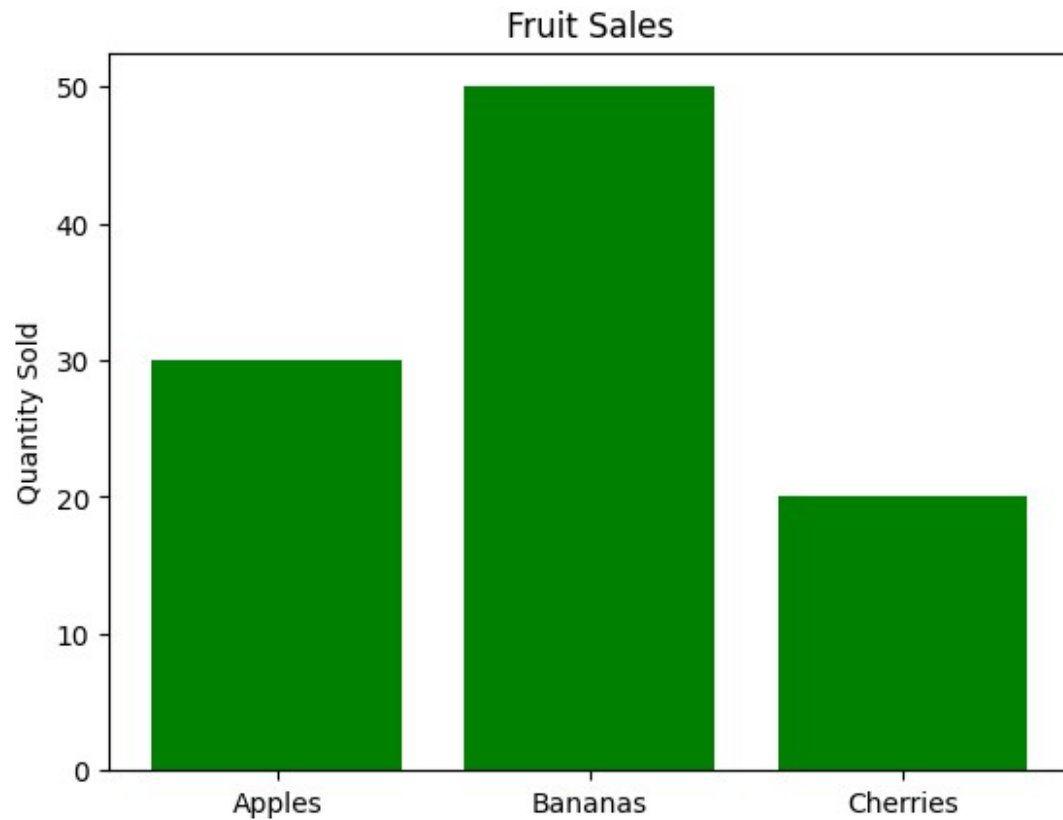


1. Bar Chart Use for: Category comparison.

Function: plt.bar() or plt.barh() (horizontal)

```
categories = ['Apples', 'Bananas', 'Cherries']
values = [30, 50, 20]

plt.bar(categories, values, color='green')
plt.title("Fruit Sales")
plt.ylabel("Quantity Sold")
plt.show()
```

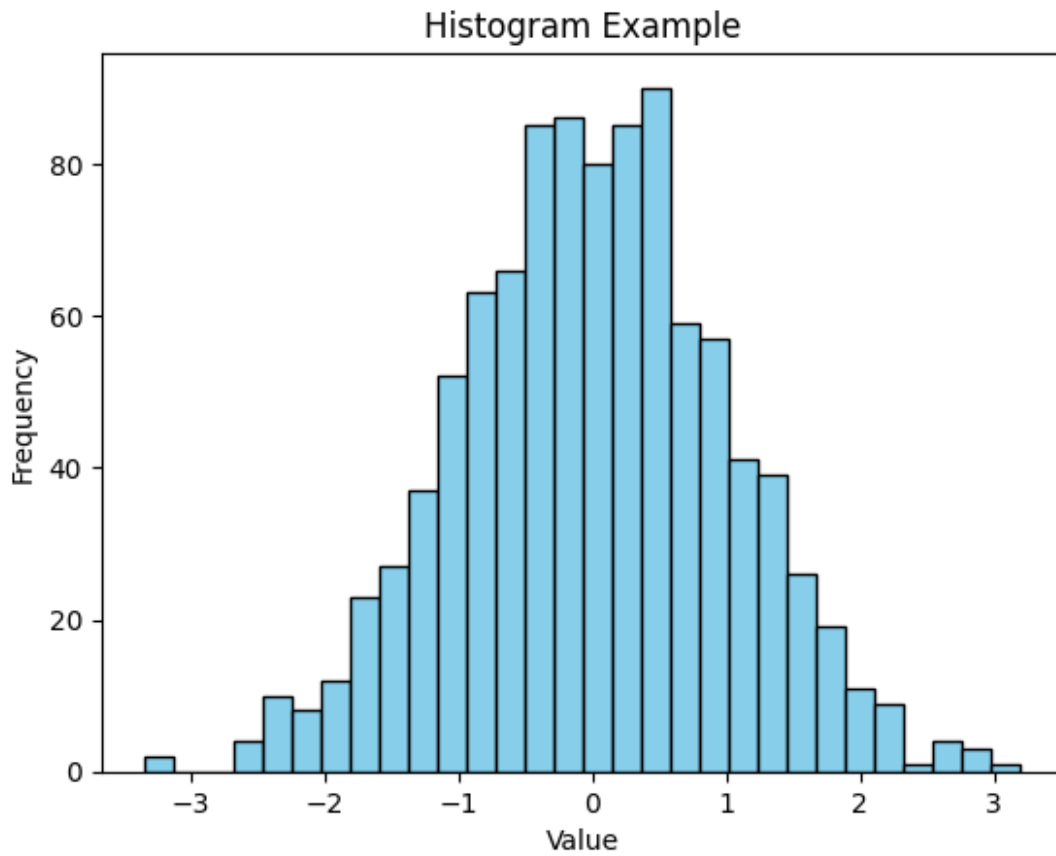


1. Histogram Use for: Distribution of numeric data.

Function: plt.hist()

```
data = np.random.randn(1000)

plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



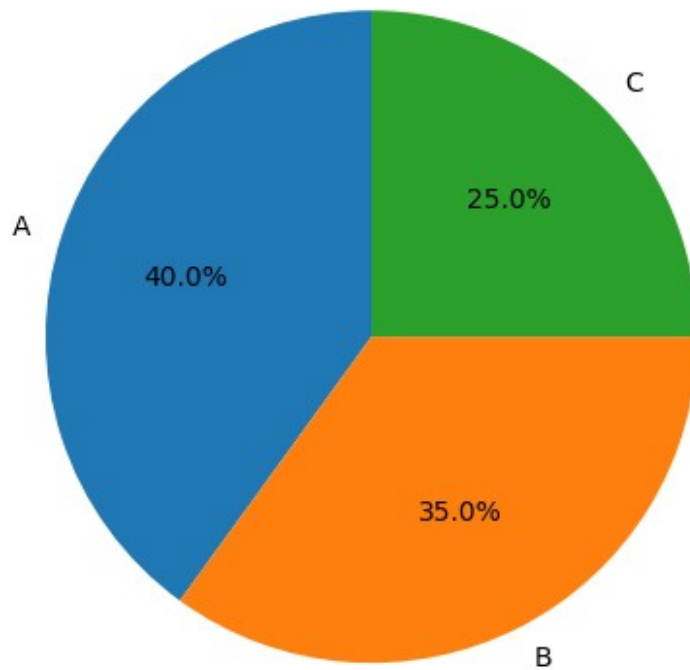
1. Pie Chart Use for: Proportion, percentage distribution.

Function: plt.pie()

```
labels = ['A', 'B', 'C']
sizes = [40, 35, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title("Pie Chart Example")
plt.axis('equal') # Ensures it's a circle
plt.show()
```

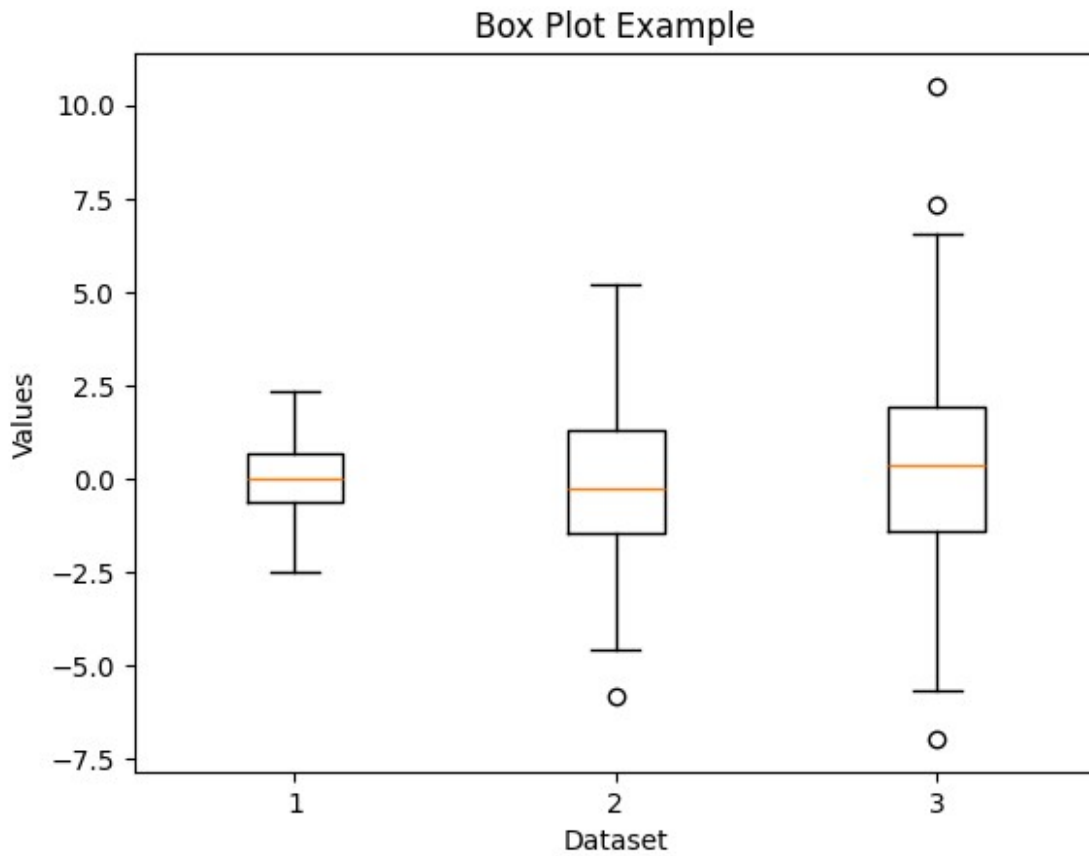
Pie Chart Example



1. Box Plot Use for: Spread, outliers, distribution shape.

Function: plt.boxplot()

```
data = [np.random.normal(0, std, 100) for std in range(1, 4)]  
  
plt.boxplot(data)  
plt.title("Box Plot Example")  
plt.xlabel("Dataset")  
plt.ylabel("Values")  
plt.show()
```

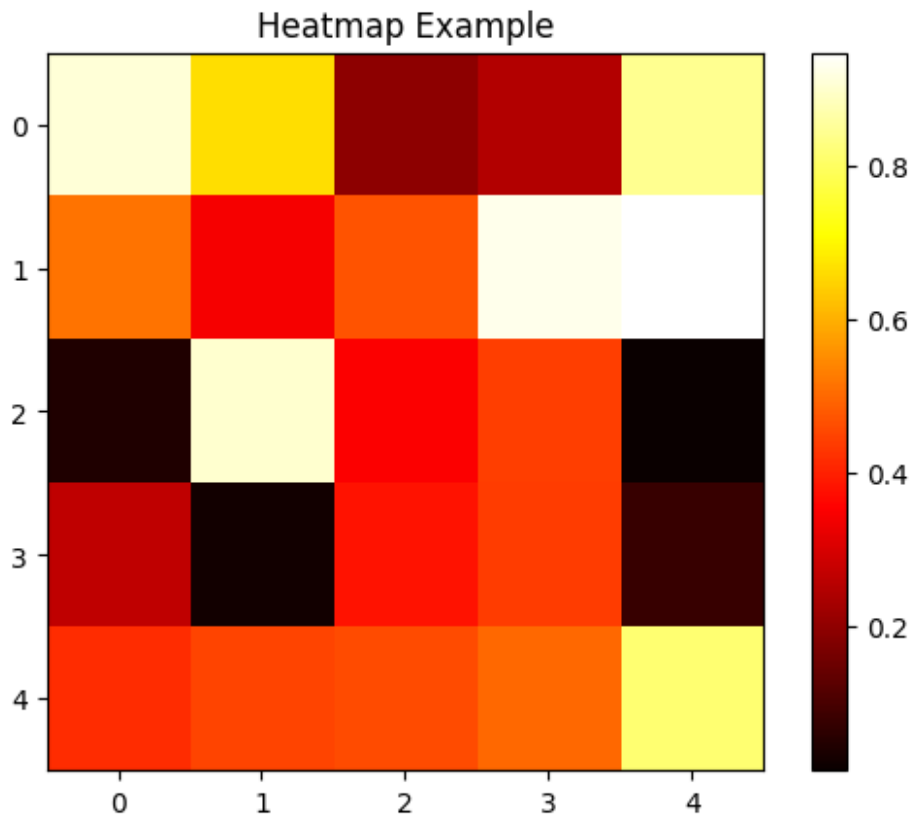


1. Heatmap (with imshow) Use for: Correlation matrices, grid heatmaps.

Function: plt.imshow()

```
matrix = np.random.rand(5, 5)

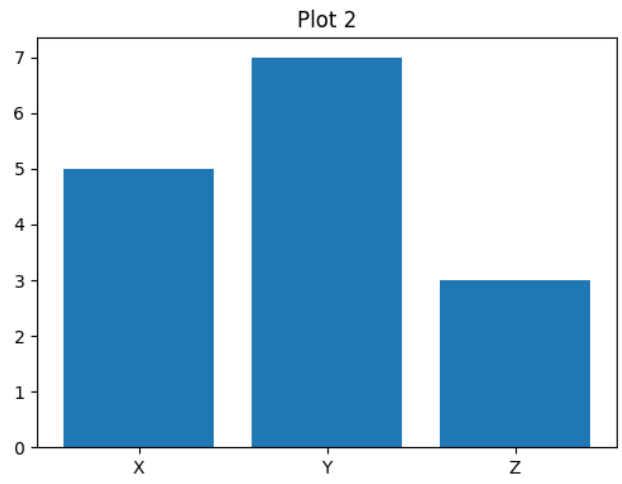
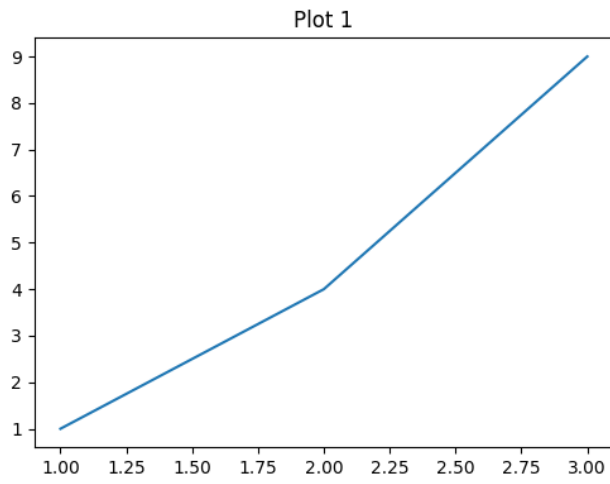
plt.imshow(matrix, cmap='hot', interpolation='nearest')
plt.colorbar()
plt.title("Heatmap Example")
plt.show()
```



1. Subplots Use for: Multiple plots in a single figure.

Function: `plt.subplot()` or `plt.subplots()`

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))  
  
axs[0].plot([1, 2, 3], [1, 4, 9])  
axs[0].set_title("Plot 1")  
  
axs[1].bar(['X', 'Y', 'Z'], [5, 7, 3])  
axs[1].set_title("Plot 2")  
  
plt.tight_layout()  
plt.show()
```



Seaborn Library Seaborn works best with DataFrames and integrates tightly with Pandas.

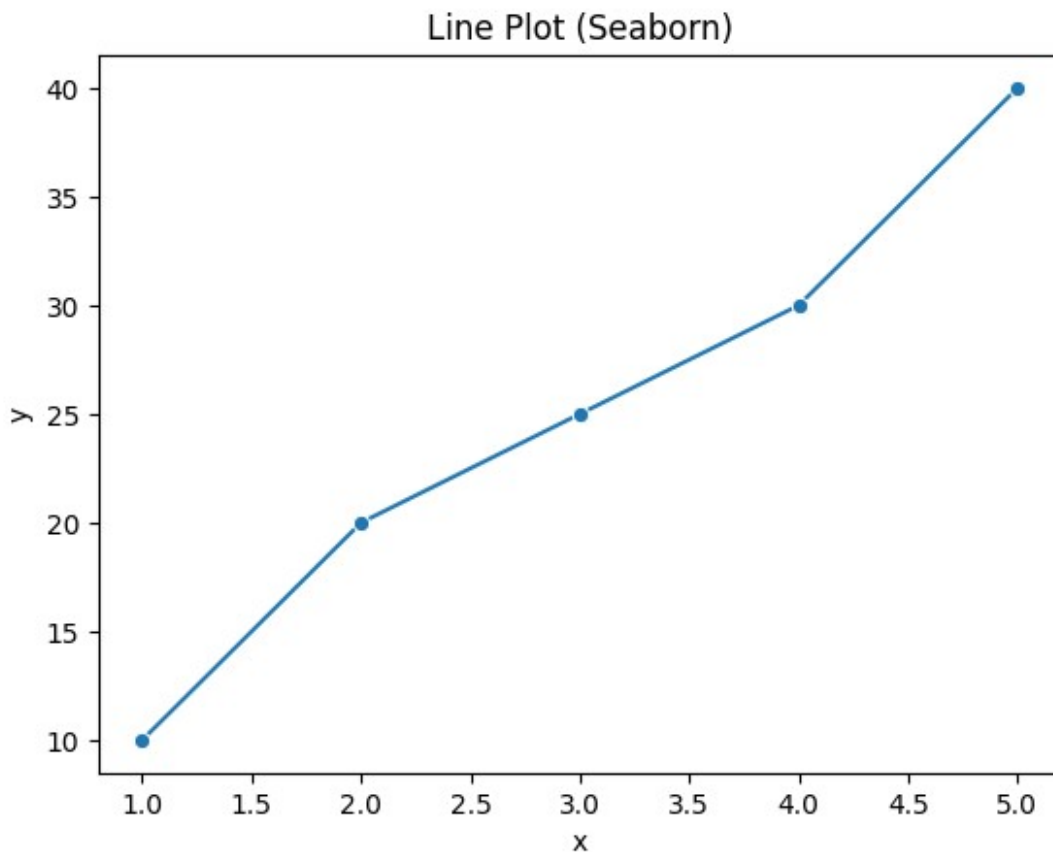
```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

1. Line Plot Use for: Trends over continuous data.

Function: `sns.lineplot()`

```
df = pd.DataFrame({
    'x': [1, 2, 3, 4, 5],
    'y': [10, 20, 25, 30, 40]
})

sns.lineplot(data=df, x='x', y='y', marker='o')
plt.title("Line Plot (Seaborn)")
plt.show()
```

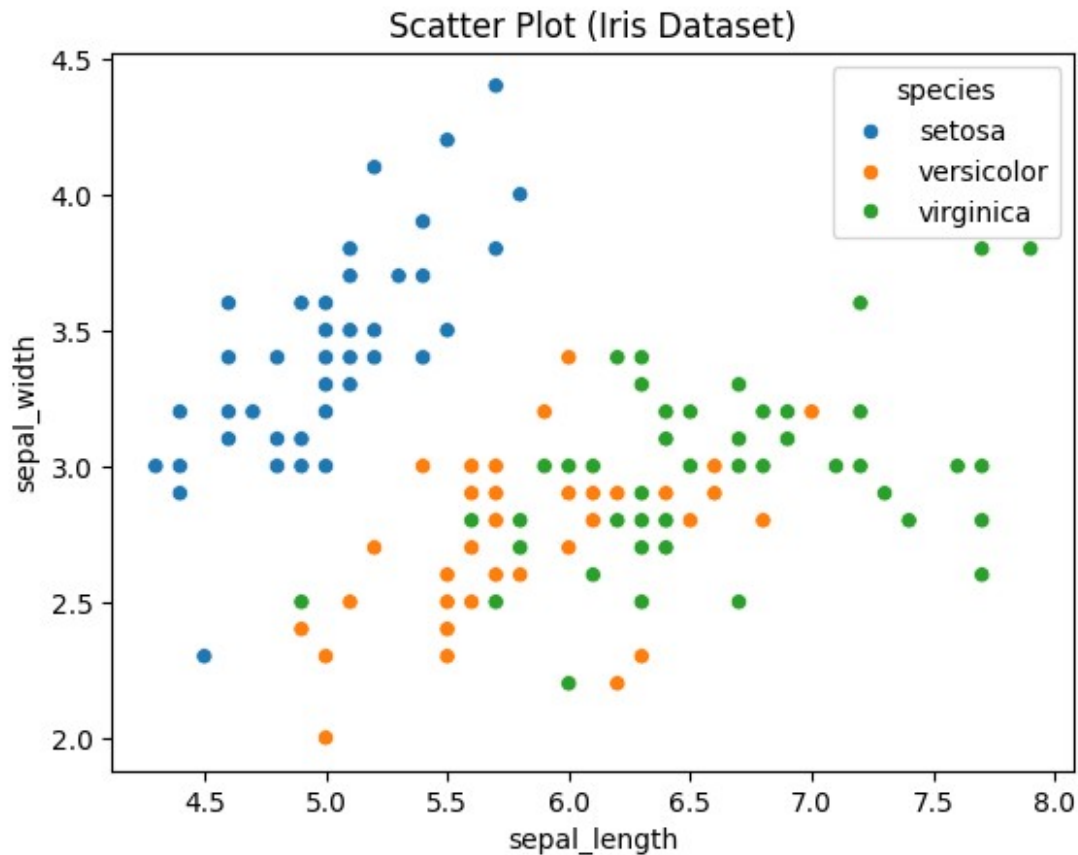


1. Scatter Plot Use for: Correlation, clustering.

Function: `sns.scatterplot()`

```
df = sns.load_dataset('iris') # Built-in dataset

sns.scatterplot(data=df, x='sepal_length', y='sepal_width',
hue='species')
plt.title("Scatter Plot (Iris Dataset)")
plt.show()
```

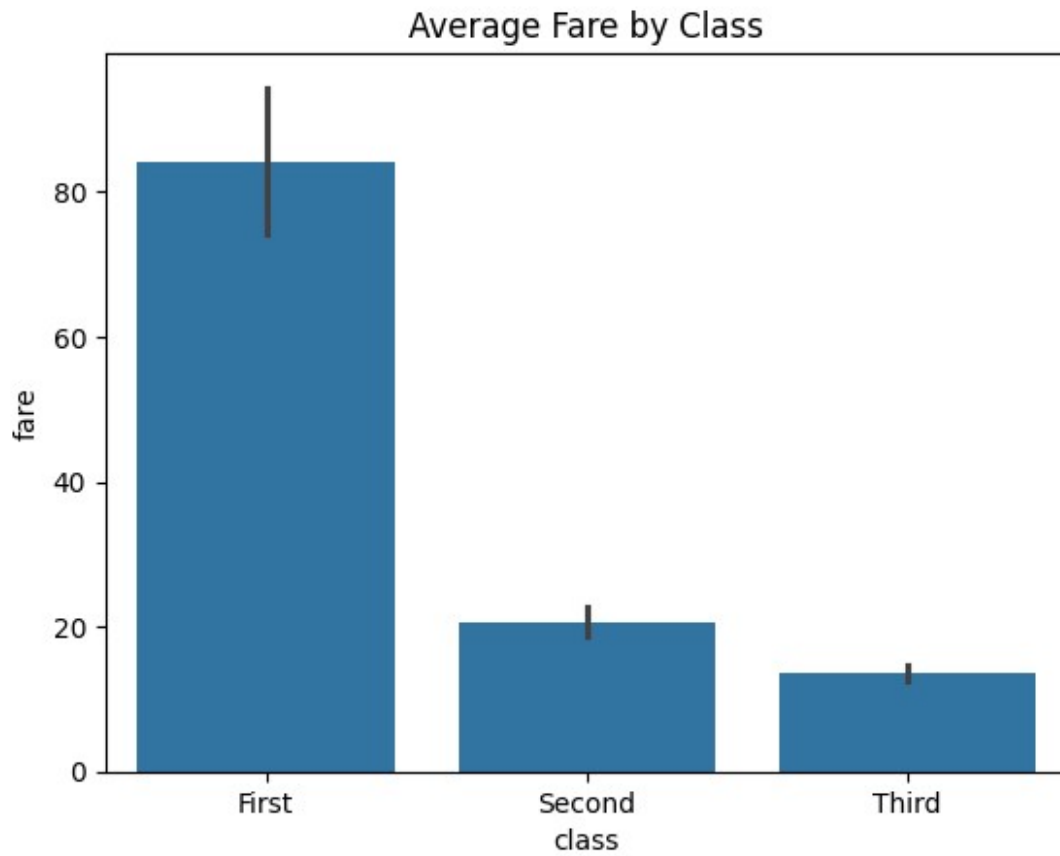


1. Bar Chart Use for: Category-wise aggregation (e.g., mean).

Function: sns.barplot() (aggregates), sns.countplot() (counts)

```
df = sns.load_dataset('titanic')

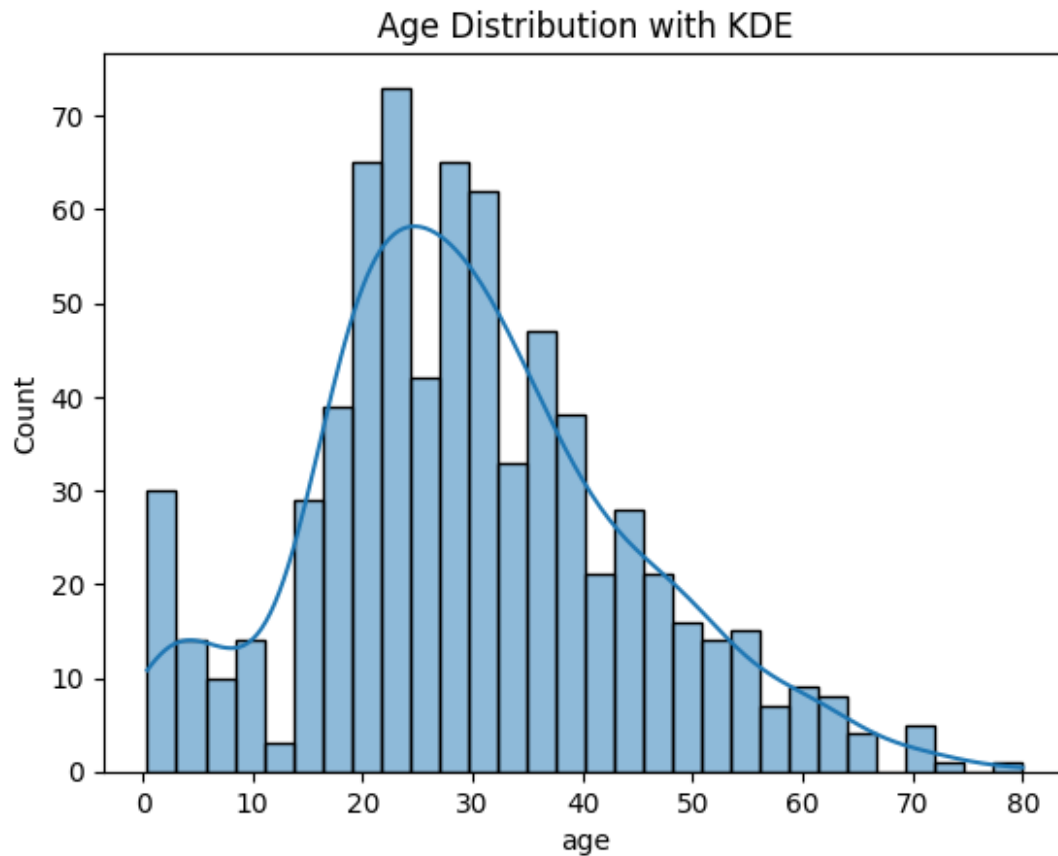
sns.barplot(data=df, x='class', y='fare')
plt.title("Average Fare by Class")
plt.show()
```



1. Histogram & KDE Use for: Distribution plots.

Function: `sns.histplot()`, `sns.kdeplot()`

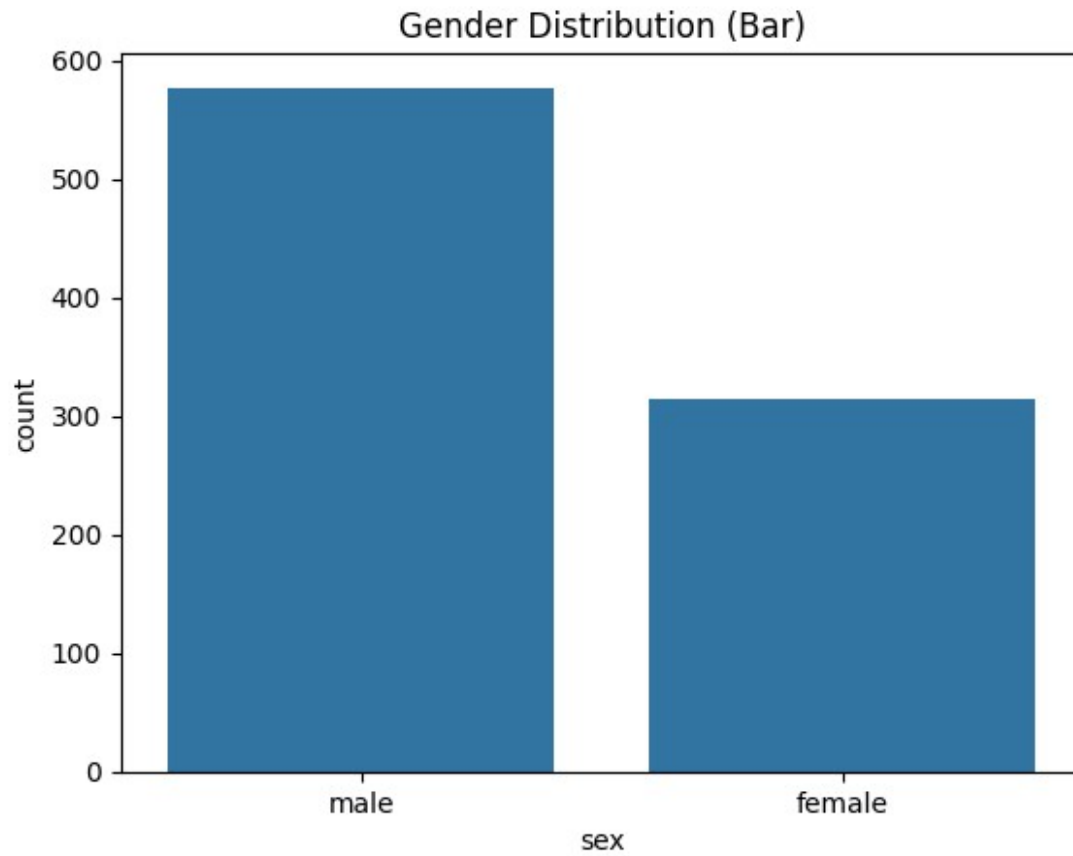
```
# Histogram
sns.histplot(data=df, x='age', bins=30, kde=True)
plt.title("Age Distribution with KDE")
plt.show()
```



1. Pie Chart (Not directly supported) □ Seaborn does not have a native pie chart — use Matplotlib for that.

Alternative (counts via bar):

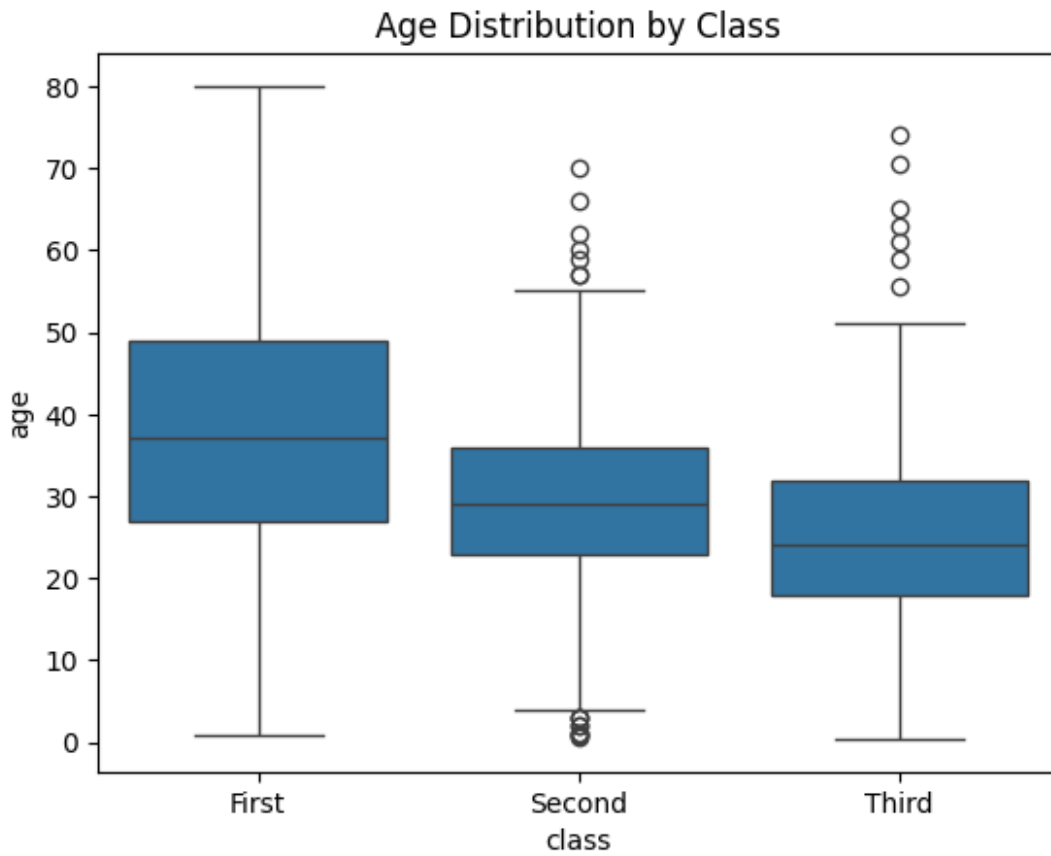
```
sns.countplot(data=df, x='sex')  
plt.title("Gender Distribution (Bar)")  
plt.show()
```



1. Box Plot Use for: Spread, outliers.

Function: `sns.boxplot()`

```
sns.boxplot(data=df, x='class', y='age')  
plt.title("Age Distribution by Class")  
plt.show()
```



1. Violin Plot Use for: Distribution + KDE + box plot in one.

Function: `sns.violinplot()`

```
sns.violinplot(data=df, x='class', y='age', hue='sex', split=True)
plt.title("Violin Plot - Age by Class and Gender")
plt.show()
```



1. Heatmap Use for: Correlation matrix or grid visualization.

Function: `sns.heatmap()`

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

1. Pair Plot Use for: Multi-dimensional exploration.

Function: `sns.pairplot()`

```
iris = sns.load_dataset('iris')
sns.pairplot(iris, hue='species')
plt.suptitle("Pair Plot (Iris)", y=1.02)
plt.show()
```

Pair Plot (Iris)

